

서찬진 · ChanJin Seo

Email scj1507@gmail.com

GitHub github.com/seochanjin

PORTFOLIO

AI 기능을 상태·실패·배포·관측성까지 연결해 운영 가능한 서비스로 구현합니다.

01

AI Pipeline & Backend

RAG · Embedding · Async Processing · State Management · Failure Handling

02

Platform & Operations

Docker · K3s · GitHub Actions · Prometheus · Grafana · Troubleshooting

03

AX & Automation

Workflow Design · Human Approval · Agent-assisted Development

AI 서비스 설계·구현

K3s 기반 서비스 배포·운영

로그·지표 기반 검증과 개선

프로젝트 요약 및 목차

문제 정의부터 설계·구현·운영·회고까지 이어진 3개의 프로젝트를 정리했습니다.

01 NewsLab2026.05–2026.07 · 이후 운영/유지보수 · 개인 Project 한 줄 설명 여러 뉴스 기사를 Topic 단위로 묶고 요약해 제공하는 장기 운영 AI 뉴스 플랫폼 본인 역할 AI Pipeline · Backend API · K3s Operations Monitoring · Troubleshooting 대표 성과 • DB 업무 지표 15종으로 부분 실패 추적 • 선정 기사만 원문 추출 · 기존 임베딩 재사용 • 100 VU·30초 p50 1.22초 → 26.05ms FastAPI · PostgreSQL/pgvector · Next.js · Docker · K3s · GitHub Actions · Prometheus/Grafana AI BackendK3s 운영관측성		
02 NotebookLM Clone2026.03–2026.04 · 개인 Project 한 줄 설명 PDF 분석부터 Hybrid RAG 답변과 근거 저장까지 구현한 문서 기반 AI 서비스 본인 역할 Spring Boot–FastAPI 책임 분리 문서 상태 · Reference · AiCallLog · RAG Pipeline 설계 대표 성과 • PROCESSING · COMPLETED · FAILED 기반 문서 상태 관리 • pgvector MMR · Keyword Search · Local Reranker 결합 • Quota 최종 실패 시 Partial Vector Cleanup 적용 Spring Boot · FastAPI · PostgreSQL/pgvector · Redis · Gemini API · Docker Compose · Nginx RAG처리 상태 관리실패 복구		
03 ACCIO2025.10–2025.11 · 5인 Team Project 한 줄 설명 자연어 상품 검색과 대화형 탐색 경험을 구현한 AI 쇼핑 서비스 본인 역할 Elasticsearch 검색 랭킹 · Frontend State Management · Guest Cart 병합 · 인증 흐름 개선 대표 성과 • 화면 이동·새로고침 이후 AI 대화와 검색 결과 복원 • 병합 실패 격리로 로그인 흐름 보호 • 비회원 Cart와 로그인 사용자 Cart 병합 React · TypeScript · FastAPI · Elasticsearch · MongoDB · Redis · AWS Bedrock AI UX상태 복원세션 관리		

여러 기사를 하나의 뉴스 흐름으로 재구성하는 AI 플랫폼

기간	형태	Live Service	GitHub
2026.05-2026.07 이후 운영/유지보수	개인 프로젝트 실제 운영	https://newslab.ai.kr	backend frontend

기획 배경

K3s를 학습하며 연결한 노드에서 직접 활용할 서비스를 운영하고자 세계 뉴스 수집 기능 개발.
이후 원문 목록만 제공하는 방식의 한계를 느껴 주요 내용을 확인할 수 있는 요약 기능 추가.

문제 정의

원문을 모으는 것만으로는 기존 뉴스 사이트와 차별성이 부족하고, 영문 기사를 개별적으로 읽어야 하는 부담이 남음. 여러 매체의 관련 기사를 묶어 주요 뉴스 흐름을 확인할 수 있는 방식 필요.

핵심 가설

제목과 RSS 요약을 Embedding해 의미적으로 유사한 기사를 묶고, 반복 보도량을 주목도 신호로 활용하면 기사를 Topic 단위 뉴스 흐름으로 재구성할 수 있다고 판단

목표

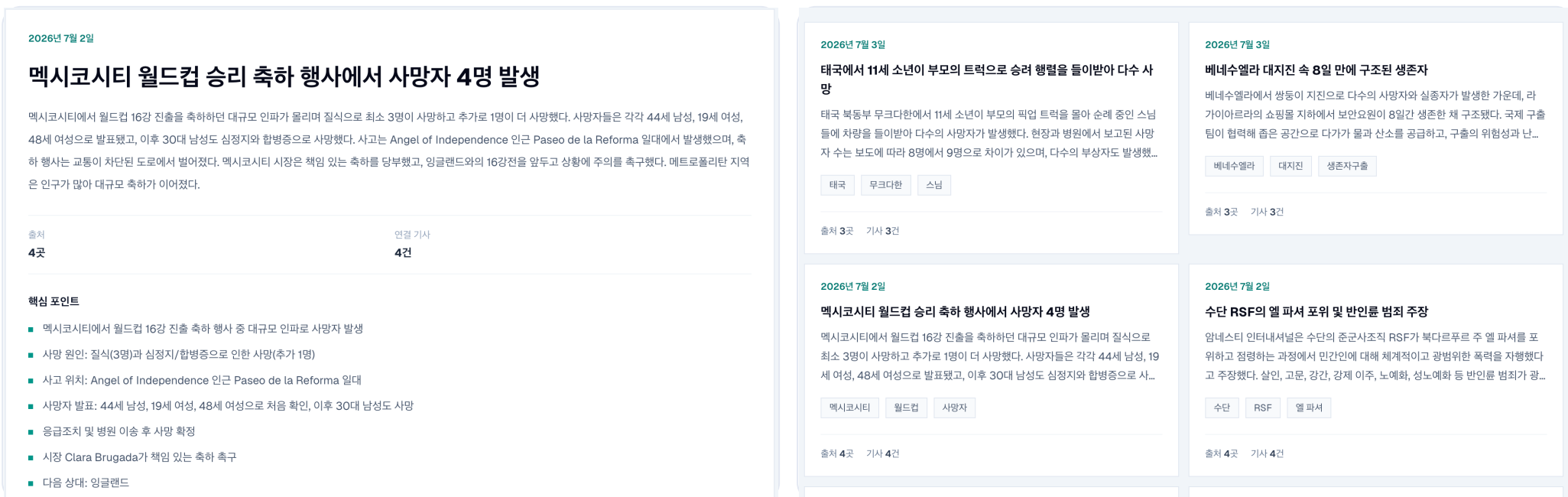
선정된 기사에만 원문 추출과 LLM 요약을 수행해 비용과 실패 범위를 제한하면서 Daily·3일·주간 뉴스 흐름을 제공

기술 스택

Platform	Docker · K3s · Traefik · Argo CD · GitHub Actions · Prometheus/Grafana · Alertmanager
Backend/Data	Python · FastAPI · PostgreSQL/pgvector
Frontend	Next.js · TypeScript



홈 화면 · 주요 이슈 · 최근 흐름



상세 페이지 · 핵심 포인트 · 키워드

토픽 페이지 · 관련 기사 모음

The diagram illustrates a cloud-native architecture managed by Tailscale, deployed on Oracle Cloud Infrastructure (OCI). The architecture is organized into several nested layers and components:

- External Access:** A **user** interacts with the system via **DNS** and an **Internet Gateway**.
- Tailscale Management:** The entire infrastructure is managed by **Tailscale**, which provides access to the **Oracle Cloud Infrastructure (OCI)**.
- Oracle Cloud Infrastructure (OCI) Layers:**
 - VCN (Virtual Cloud Network):** Contains the **Public Subnet**.
 - Public Subnet:** Houses the **master** and **worker** nodes.
- Kubernetes Master Node:**
 - Pod cert-manager** and **Pod x 7 Argo CD** are deployed.
 - Pod traefik** acts as the ingress controller.
 - K3s server** components include **API Server**, **Scheduler**, **Controller Manager**, and **SQLite**.
 - k3s 기본 add-on:** CoreDNS, metrics-server, and local-path-provisioner.
- Kubernetes Worker Node:**
 - Pod x 2 NEXT.js** and **Pod x 2 FastAPI** are deployed.
 - Pod redis** is used for caching.
 - CronJob x 4 FastAPI** is scheduled for specific times (03:00 / 04:00 / 05:00 / 월 00:30).
 - Additional add-ons:** cert-manager(cainjector), webhooks, kube-prometheus-stack, node-exporter, and svclb-traefik.
- PI-Node (Pi-Node):**
 - DaemonSet:** node-exporter, svclb-traefik.
 - NoSchedule taint — toleration 명시 workload만 배치** (NoSchedule taint — explicitly specify toleration to schedule only workload).
- External Services:** The application interacts with **OpenAI**, **supabase** (including **PostgreSQL** and **pgvector**), and **redis**.

노드 배치 판단

- master — control plane + 클러스터 add-on
- worker — 앱 전용 (workload=app)
- Pi — NoSchedule, 관측만

가정 회선·전원 의존 → 전달 보장 워크로드 제외
worker memory request 12% → 확장 불필요

운영 경계

- 조화·진단 — Agent 보조
- Git 관리 대상 — Argo CD manual sync, 사람 승인
- Secret·monitoring·rollout — 사람이 직접

모든 기사에 고비용 처리를 하지 않는 단계형 AI Pipeline

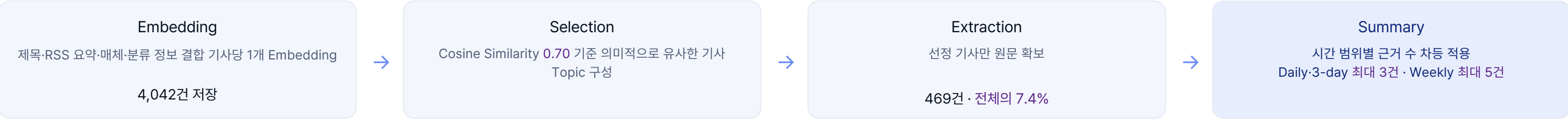
문제

- 모든 기사 원문을 선추출하면 외부 요청 수와 실패 지점이 기사 수에 비례해 증가
- Daily·3-day·Weekly 실행마다 Embedding을 다시 만들면 동일 기사에 비용과 처리시간이 반복 발생
- 기사 단위 오류를 격리하지 않으면 일부 실패 때문에 해당 실행 전체 결과가 영향받을 수 있음

설계 결정

- 제목·RSS 요약·매체·분류 정보를 결합해 기사당 1개 Embedding을 생성하고 Source Hash로 재사용
- 운영 기사 샘플에서 Cosine Similarity Threshold 0.60~0.80을 비교해 운영값 0.70 선택
- 시간 범위별 기사량에 따라 Summary 근거 수를 차등 적용 Daily·3-day 최대 3건 · Weekly 최대 5건
- Stage별 실패 격리로 부분 실패가 전체 Batch로 번지지 않게 설계

Stage 분리 — 전체 기사에서 근거까지, 실패 격리 단위



수집 6,372건 → Embedding 4,042건 → 원문 추출 469건(7.4%) → Summary 232회

운영 지표 기준: 2026.07.19

Daily CronJob 산출물을 3-day · Weekly가 재사용

Daily CronJob · 04:00

매일 실행

기사 메타데이터 Embedding (DB 저장) → Selection → Extraction → Summary

Daily Summary 근거 최대 3건

3-day·Weekly는 기존 Embedding을 재사용하고 Selection · Extraction · Summary 단계만 다시 실행합니다.

3-day

매일 실행

최근 72시간 기사를 기존 Embedding으로 재클러스터링 3-day 전용 테이블과 API에 저장 Summary 근거 최대 3건

Weekly

주 1회 실행

직전 완료 주간 기사를 기존 Embedding으로 재클러스터링 Weekly 전용 테이블과 API에 저장 Summary 근거 최대 5건

NewsLab 운영 장애 진단 및 대응

Networking · Batch · Memory 세 가지 사례의 증상, 판단, 조치, 검증을 정리

Networking

Ready인데 kubectl logs 실패

상황

유휴 Raspberry Pi를 K3s Worker로 추가해 Cloud와 Home Lab을 결합한 Hybrid Cluster 구성

문제

OCI Private Network와 가정 LAN의 경계가 다르고 Node가 Private·LAN·Tailscale 주소를 혼용

판단·조치

소규모 3-Node 환경에서 설정이 단순한 Tailscale을 선택하고 node-ip / flannel-iface를 Tailscale 주소로 통일

검증

양쪽 Worker 로그 확인 Pi에 NoSchedule Taint 적용

Evidence: docs/devlog/infra-pi-worker-join.md

Batch

Daily Pipeline 5시간 이상 Running

상황

Daily Pipeline이 5시간 이상 Running 상태로 남음

한계

재실행에서는 정상 완료되어 동일 현상을 재현하지 못했고 확정 원인은 특정하지 못함

조치

concurrencyPolicy: Forbid activeDeadlineSeconds: 1800 backoffLimit: 1 Stage Progress Log 추가

효과

중복 실행과 장시간 실행의 상한을 설정하고 재발 시 정체 Stage를 확인할 진단 경로 확보

Evidence: k8s/news-daily-topic-pipeline-cronjob.yaml

Memory

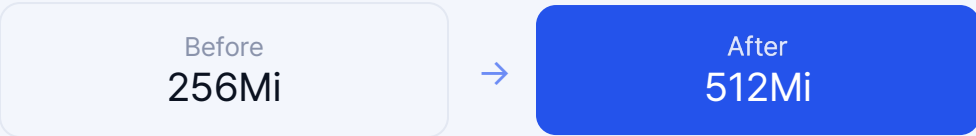
Grafana OOMKilled · Exit 137

배경

제한된 Worker 자원에서 Monitoring Core를 운영하고 Prometheus 데이터 보관 기간을 1일로 제한

판단·조치

Container Memory Limit 초과로 판단



검증

3/3 Running · Restart 0

Evidence: k8s/monitoring/kube-prometheus-stack-values.yaml

노드와 파드는 정상인데 서비스는 504

Tailscale 재시작 후 노드 간 통신 단절을 추적하고, 워커의 복구 동작을 검증

장애 대응 흐름

01 증상 서비스 504와 배치 실패

공개 서비스 응답 불가, CronJob 4종 실패



02 진단 노드 간 통신 범위 비교

같은 노드에서는 성공, 다른 노드에서는 실패



03 조치 복구 후 재시작 의존 관계 설정

K3s 재시작 등으로 통신과 공개 서비스 복구

systemd 설정: After / Wants / PartOf



04 검증 워커에서 재시작 상황 재현

Tailscale 재시작 후 K3s 자동 재시작과 통신 복구 확인

01 배경

- K3s 노드 간 통신에 Tailscale을 사용해 뉴스 서비스 운영
- 공개 API와 기사 수집·요약용 CronJob 4종을 클러스터에서 실행

02 문제

- 노드 Ready·파드 Running 상태에서도 서비스 504와 CronJob 4종 실패
- 같은 노드 통신은 성공하고 다른 노드 통신은 실패
- Tailscale 재시작 시점의 오류 로그와 flannel.1 부재 확인

03 설계 판단

- 일회성 복구에 그치지 않고 Tailscale과 K3s의 재시작 연계 필요
- After·Wants로 기동 순서·의존 관계, PartOf로 재시작 전파 설정
- 실행 상태 외에 노드 간 통신과 실제 서비스 응답까지 확인

04 실제 적용과 결과

- K3s 서비스와 애플리케이션 재시작 후 공개 서비스 HTTP 200 확인
- 수집·요약 작업을 순차 재실행해 처리 공백 보완
- 워커에서 Tailscale을 재시작해 k3s-agent 자동 재시작 확인
- flannel.1 복원 및 DNS 서비스 도달 확인

05 한계와 검증

- 워커에서 재시작 대응 검증, 마스터 재현 훈련은 미수행
- Tailscale 자체 재시작의 직접 원인과 다른 단절 상황은 미확인

한계: 총 중단 25시간 23분 중 감지 지연 24시간 40분. 재시작 대응 검증은 워커에서 수행.
Tailscale 재시작 자체의 직접 원인은 미확인. 다른 원인의 인터페이스 단절까지 검증한 것은 아님.

반복 조회 부하를 캐시 계층으로 분리한 Home API

동일한 100 VU·30초 조건에서 캐시 적용 전후의 응답 지연 비교

p50 (100 VU)

1.22s → **26.05ms**

RPS 증가율 (50→100 VU)

39% → **97%**

단계별 측정 (Before → After)

VU	RPS	p50	p95
1	0.63 → 0.94	533ms → 19ms	765ms → 36ms
10	6.15 → 8.82	516ms → 18ms	685ms → 33ms
25	13.87 → 20.68	520ms → 20ms	1.51s → 93ms
50	29.07 → 43.88	517ms → 22ms	1.35s → 266ms
100	40.50 → 86.59	1.22s → 26ms	2.20s → 412ms

cache-aside 흐름

요청 → Redis 조회

└ hit → 응답

└ miss → PostgreSQL 조합 → Redis 저장 시도 → 응답

저장 실패·timeout 50ms 초과에도 응답은 계속 (fail-open)

k6 fixed-VU 30s · 각 단계 cold-cache 시작 · 측정 endpoint는 동일 Ingress의 대체 호스트 api.dev-scj.site

01 배경

- Home API — Daily·3-day·Weekly Topic을 매 요청마다 PostgreSQL에서 조합
- Topic은 CronJob 실행 시점에만 갱신, 조회는 훨씬 빈번

02 문제

- 1~50 VU 구간 p50 516~533ms로 평탄
- 100 VU에서 p50 1.22초 · p95 2.20초 · p99 3.25초로 증가
- 50→100 VU RPS 증가율 약 39%, 응답 지연 동반

03 설계 판단

- PostgreSQL은 source of truth 유지, Redis는 삭제 가능한 캐시 계층으로 한정
- fail-open — 미설정·연결 실패·timeout·payload 손상 전부 PostgreSQL fallback
- 최신성은 TTL이 아닌 prewarm — Pipeline 저장 직후 동일 builder로 key overwrite

04 실제 적용과 결과

- 100 VU — p50 1.22초 → 26.05ms, RPS 40.50 → 86.59
- 50→100 VU RPS 증가율 39% → 97%, 해당 조건에서 처리량 증가
- p99는 3.25초 → 2.25초로 개선폭 상대적으로 작음
- 전 단계 HTTP 오류 0건 · Pod restart 0건

05 한계와 다음 검증

- 각 단계 cold-cache 시작 → 10·50 VU p99가 Before보다 높음
- 요청별 trace·DB 지표 없어 원인 미확정
- 30초 단발 측정 — 장시간 부하 hit rate 추이 미확인

캐시가 죽어도 API가 죽지 않는 구조를 실제로 확인

Production Redis를 중단하고 PostgreSQL fallback과 복구 후 재생성을 순서대로 검증

중단 → 복구 검증 순서

① 정상 Redis 1/1 EXISTS=1	② Redis 중단 replicas=0 · Pod 없음 HTTP 200 · 846ms items 10 · restart 0 fail-open 확인	③ 복구 replicas=1 · EXISTS=0 miss → store → hit 49ms
---	---	--

fallback 경로

요청 → Redis 조회 timeout 50ms
└ hit → 응답
└ miss → PostgreSQL → 저장 시도 → 응답
└ 오류·timeout → PostgreSQL → 응답
캐시 실패가 요청 실패로 이어지지 않습니다

단위 테스트로 검증한 실패 경로

GET timeout	SET 실패
connection 오류	손상된 payload
malformed URL	unsupported scheme
TTL 만료	Redis 미설정

회귀 테스트 422 passed · Argo CD Synced/Healthy · restart 0 · 검증 시점 TTL 60초
(현재 운영값 Daily·3-day 108,000초 / Weekly 691,200초)

Redis 중단·복구는 사람이 승인해 직접 실행했으며 Agent는 production mutation을 실행 x

01 배경

- Redis 도입 → Home API가 캐시 계층에 의존
- "캐시 없어도 응답한다"는 코드에만 있던 전제 — 중단시켜 보기 전까지는 가정

02 문제

- Redis 장애 형태 다양 — 연결 실패 · timeout · payload 손상 · URL 설정 오류
- 각 경우가 예외로 전파되면 캐시가 오히려 장애 지점을 늘림

03 설계 판단

- fail-open — 모든 Redis 오류를 PostgreSQL fallback으로 격리
- connect·read timeout 50ms — 캐시 지연이 응답 지연으로 전이되지 않게 제한
- persistence 없이 운영 — 유실 시 PostgreSQL에서 재생성
- 단위 테스트로 실패 경로 8종 개별 검증

04 실제 적용과 결과

- 사람이 승인한 운영 창에서 replicas=0으로 Redis 중단
- Redis Pod 부재 상태에서 /topics/home — HTTP 200 · 기존 schema 유지 · items 10건
- API Pod restart 0건
- 복구 후 EXISTS=0 → miss → store(TTL 60) → hit(49ms) 순서대로 확인

05 한계와 다음 검증

- 운영 로그에 home_topics_cache event 미노출 → hit·miss·store를 key 존재와 TTL 변화로 확인
- PostgreSQL query·connection 수 미수집 → DB 부하 감소량 정량화 안 함
- cold-cache 동시 요청의 cache stampede 대응 없음

NewsLab / 운영 경험으로 넓어진 개발 관점

01 실행 상태와 실제 처리 결과

수집 작업이 성공해도 일부 기사 처리는 실패할 수 있어 DB의 단계별 처리 현황을 지표로 노출했습니다. 이를 바탕으로 대표 기사에 원문이 없으면 대체 기사가 있어도 토픽을 폐기하던 로직을 찾아 수정했습니다.

실행 성공 여부와 함께 **실제 저장 결과와 누락된 데이터도 확인**해야 한다는 점을 배웠습니다.

02 개선 후 장애 상황까지 확인

Redis 캐시로 조회 응답을 개선한 뒤, 캐시를 직접 중단해 DB 조회로 전환되는지 확인했습니다. 복구 후 캐시가 다시 생성되는 과정도 점검했습니다.

성능을 위해 추가한 구성요소가 **실패할 때의 동작까지 검증**해야 한다는 기준이 생겼습니다. 단기 부하 시험만으로 장시간 안정성을 확인한 것은 아니라는 한계도 남겼습니다.

03 알림과 백업의 실제 동작 검증

알림 3종은 발생뿐 아니라 해제 메시지까지 수신하는지 확인했습니다. DB 백업은 격리된 환경에 복구하고 데이터를 대조했습니다.

이 과정을 통해 알림 설정과 백업 파일의 존재만으로 대응 준비가 끝났다고 판단하기 어렵다는 것을 배웠습니다. 운영 기능을 마련할 때 **실제로 사용할 수 있는지 확인하는 절차**도 함께 필요했습니다.

04 이전 프로젝트와 달라진 점

문서 분석 서비스에서는 API와 처리 상태 설계에 집중했습니다. 이후 운영도 익혀보고 싶어 K3s로 노드를 연결하고 직접 사용할 뉴스 서비스를 만들었습니다. 네트워크 문제와 예약 작업 실패를 겪고, 한정된 자원에서는 임베딩 재사용과 원문 추출 대상 선별도 고민했습니다.

코드의 동작에서 **실행 환경과 자원 사용까지 고려**하는 범위가 넓어졌습니다.

문서 근거와 처리 상태를 관리하는 RAG 서비스

PDF 업로드, 비동기 분석, Hybrid RAG 검색, 답변 근거 저장을 하나의 서비스 흐름으로 구현

기간	형태	GitHub
2026.03-2026.04	개인 프로젝트	https://github.com/notebooklm-clone-sci
	4개 Repository 분리 구성	
	배포검증 NCP ·	

기획 배경

팀 프로젝트에서 경험한 RAG를 직접 설계·구현하며, Java·Spring 기반 서비스 관리와 Python·Fast API 기반 AI 처리 구조를 학습하기 위해 시작

문제 정의

PDF 분석은 처리 시간이 길어 외부 API 실패 시 문서 상태와 Vector 데이터가 어긋날 수 있음
답변 근거 미저장 시 어떤 문서를 기반으로 답변했는지 사후 확인 불가

설계 방향

Spring Boot는 사용자·문서·채팅·처리 상태를 관리

FastAPI는 PDF 파싱·Embedding·검색·LLM 호출을 담당

문서 분석은 비동기 실행하고 결과를 PROCESSING · COMPLETED · FAILED 상태로 관리

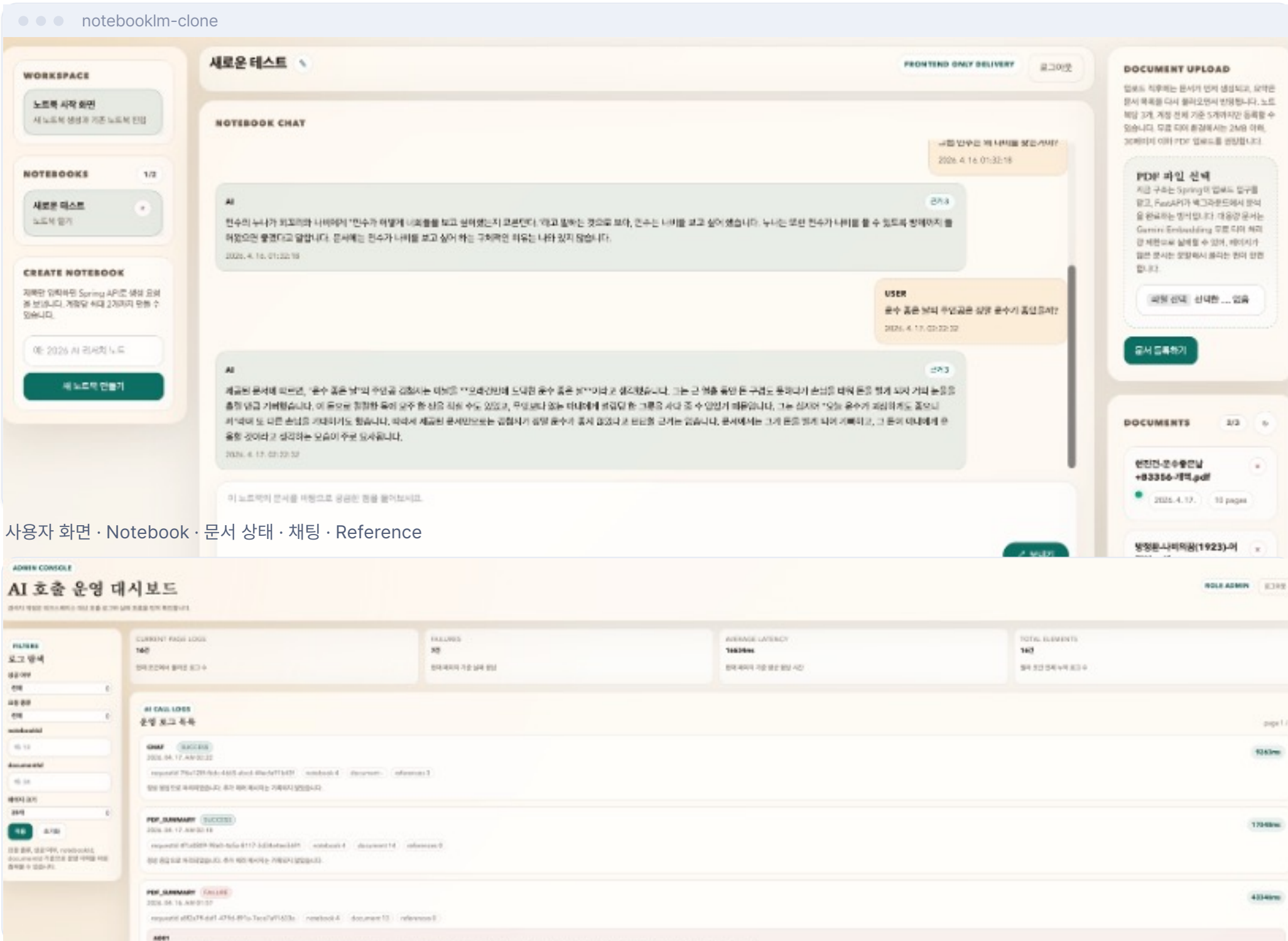
핵심 구현

Hybrid RAG 검색, 답변 Reference 저장, AI 호출 로그를 연결

문서 업로드부터 분석·검색·답변·근거 재조회까지 하나의 서비스 흐름으로 구현

기술 스택

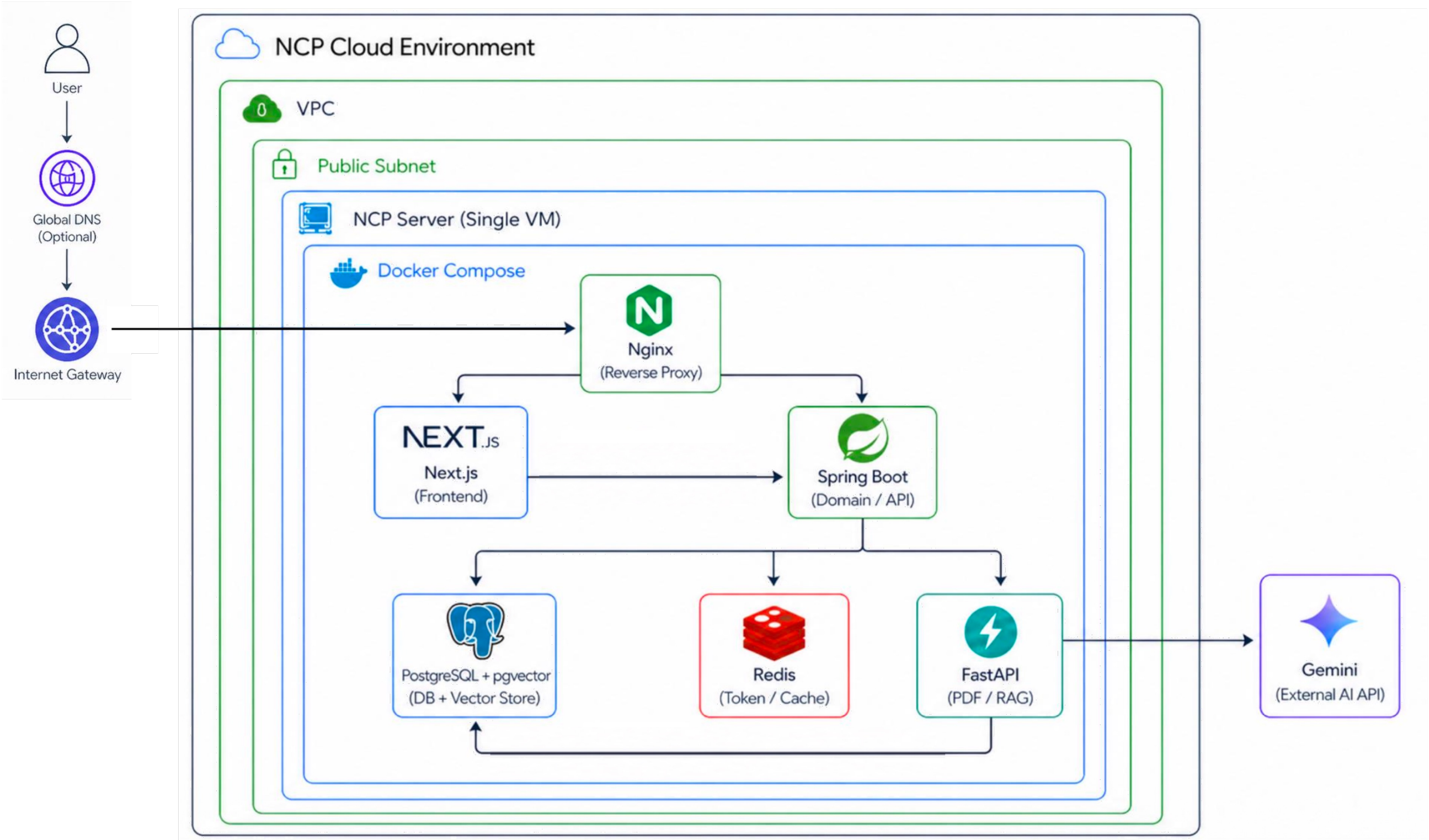
Backend	Java · Spring Boot · Python · FastAPI
AI / Data	Gemini API · PostgreSQL/pgvector · Redis · LangChain
Frontend	Next.js · TypeScript
Infrastructure	Docker Compose · Nginx · NCP



운영 화면 · requestId 기반 AiCallLog 조회

NCP 단일 VM에 Docker Compose로 배포한 Runtime 구조

4개 Repository의 애플리케이션과 운영 설정을 Nginx 진입점 아래 하나의 실행 환경으로 조립

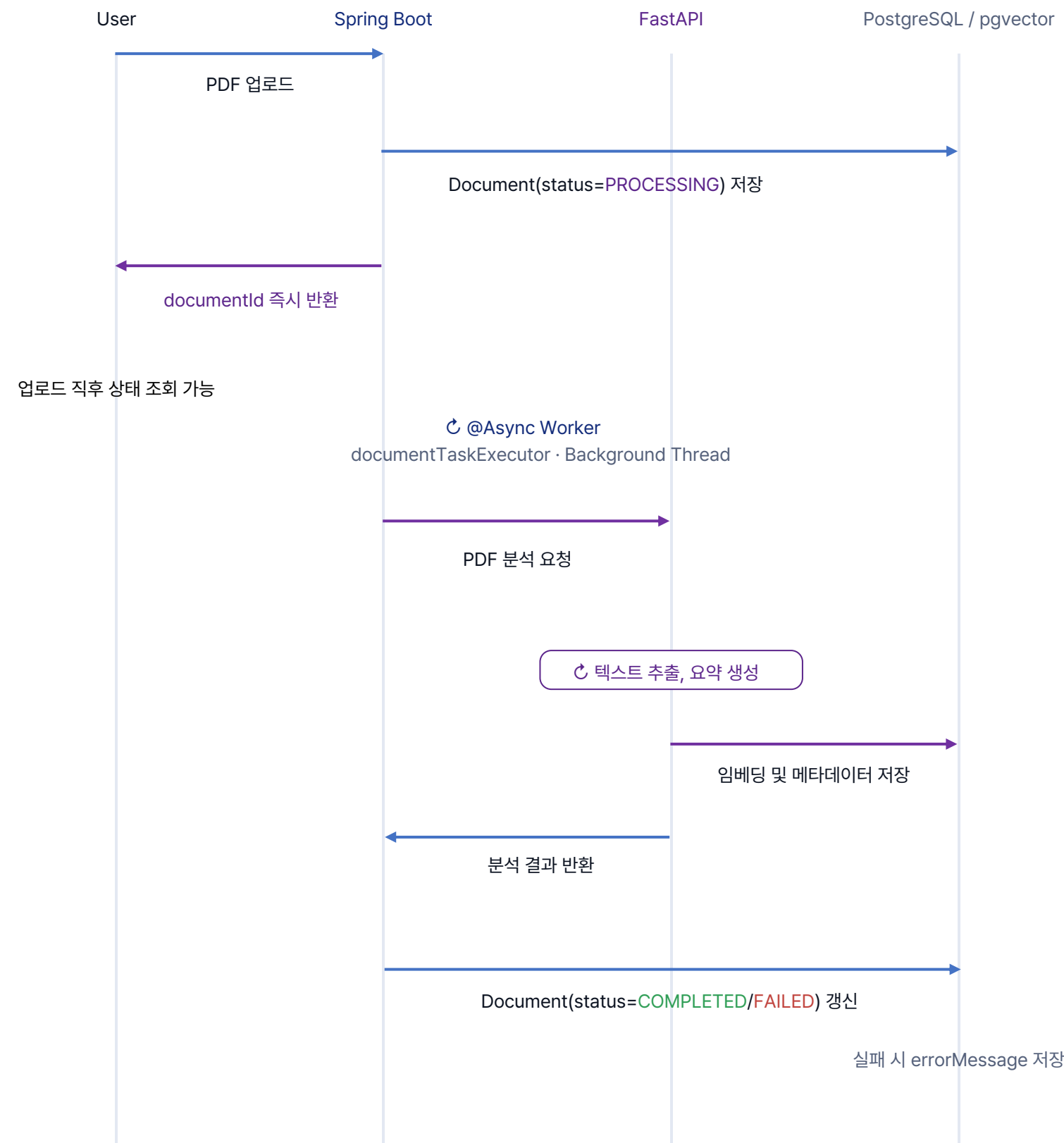


Repository → Runtime Component Mapping

core-api-spring Spring Boot · Domain / API	ai-worker-fastapi PDF · Embedding · Retrieval · LLM	frontend-ui Next.js · Chat / Reference UI	infra-config Docker Compose · Nginx · Environment
---	--	--	--

도메인 상태와 AI 처리를 분리한 비동기 문서 분석 구조

Spring Boot는 서비스 상태와 계약을 관리하고, FastAPI는 문서 분석과 AI 처리를 담당하도록 분리



01 배경

- PDF 파싱과 Embedding은 처리 시간이 길어, 분석이 끝날 때까지 업로드 요청을 유지하면 사용자 응답과 장시간 AI 처리가 결합되는 문제 발생

02 문제

- 외부 AI API 대기 중 요청 Thread·DB Transaction 장시간 유지 → 응답 지연과 Connection 점유 확대
- 처리 실패를 상태로 남기지 않으면 사용자가 분석 결과를 재확인 불가

03 설계 판단

- Spring Boot는 사용자 소유권, Document 상태, API 계약과 결과 저장을 담당
- FastAPI는 PDF 파싱, Summary, Chunking, Embedding과 Vector 저장을 담당하도록 책임을 분리
- 외부 AI API 대기 구간은 장시간 DB Transaction과 분리하고, 분석 결과를 저장할 때 필요한 DB 작업만 수행

04 실제 적용과 결과

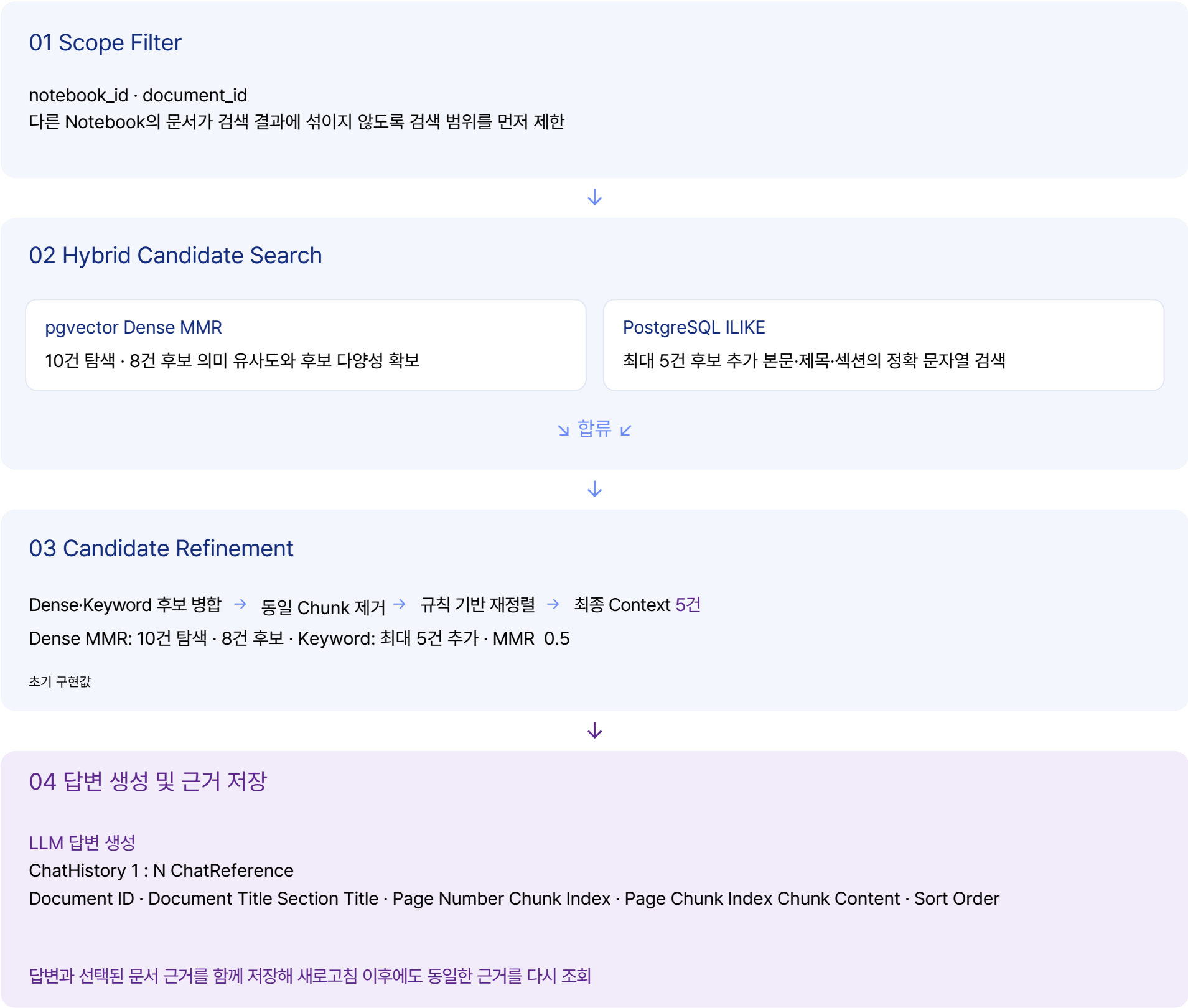
- @Async + 전용 ThreadPoolTaskExecutor — Document를 PROCESSING으로 저장 후 documentId 즉시 반환
- 별도 Worker Thread가 FastAPI 호출 — 성공 시 COMPLETED, 실패 시 FAILED + errorMessage 저장
- 업로드 직후 처리 상태 조회 가능
- 사용자 응답 시간과 AI 처리 시간 분리
- 실패 상태를 Document 데이터로 추적

05 트레이드오프와 다음 개선

- Thread Pool Core 5 · Max 10 · Queue 50 — 부하 시험으로 도출한 최적값이 아닌 초기 설정
- 동시 업로드 증가 시 실행 지표 기반으로 Pool 크기 조정 예정
- 작업 Queue와 독립 Worker 도입 → API 요청과 문서 처리 Lifecycle 분리

Hybrid Retrieval과 답변 근거 저장 구조

pgvector MMR과 키워드 검색을 결합하고, 선택된 문서 근거를 답변과 함께 저장



01 배경

- Vector 유사도 검색은 의미가 비슷한 문서 탐색에 유용
- 인물명·날짜·숫자 같은 정확한 문자열 조건은 놓칠 수 있음

02 문제

- 유사도 결과만 사용 시 비슷한 Chunk 반복, 질문과 직접 관련된 문장보다 주변 문맥이 우선될 수 있음
- 답변만 저장하면 새로고침 이후 근거 문서 확인 불가

03 설계 판단

- Notebook·Document로 검색 범위 먼저 제한
- pgvector MMR 후보 + PostgreSQL ILIKE 후보 병합
- 중복 Chunk 제거 후 규칙 기반 재정렬 — 질문 토큰 중복도 · 직접 포함 여부 · 기존 MMR 순위 조합

04 실제 적용과 결과

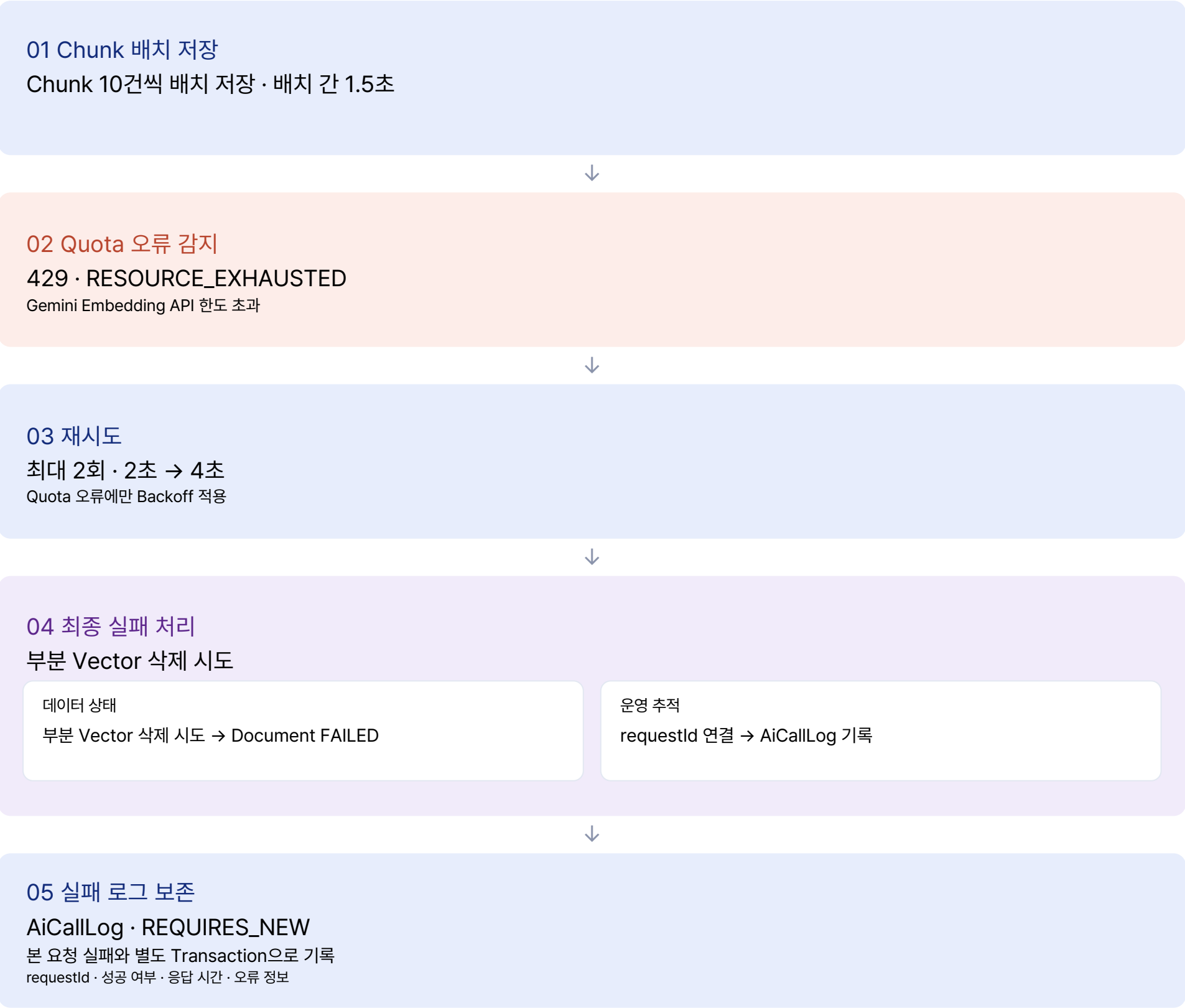
- 선택된 Chunk를 답변과 함께 ChatReference로 저장
- 문서 제목 · Section · Page · Chunk 순서 · 원문 보존
- 새로고침 이후에도 동일한 답변 근거 재조회 가능

05 한계와 다음 개선

- 재정렬이 고정 규칙 기반, 정답 데이터셋 기반 검색 품질 평가 미수행
- 평가 질문·정답 근거를 구성해 Recall·MRR 측정 예정
- 필요 시 학습형 Reranker와 비교

Quota 실패 시 부분 저장을 정리하고 호출 원인을 추적한 구조

Embedding 배치 처리, 최종 실패 시 부분 Vector 정리와 실패 상태·호출 로그 기록



NotebookLM Clone / 책임 분리와 전체 처리 흐름에서 배운 점

01 이해하고 수정하기 위한 책임 분리

학습한 Spring 자료에서는 AI 기능을 연결하는 설명이 적었고, 접한 AI 구현 자료는 주로 Python 기반이었습니다. 사용자·문서·상태 관리는 Spring Boot에, 문서 분석과 AI 처리는 FastAPI에 나누어 구현했습니다.

각 기능을 이해하고 수정하기 편해지면서, 기술 선택에 **기능의 성격과 수정 편의성**도 함께 고려하게 됐습니다.

02 요청 이후의 처리 상태 관리

문서 분석을 비동기로 실행하고 진행·완료·실패 상태를 저장했습니다. 외부 API 사용량 제한에는 재시도와 지수 백오프를 적용하고, 최종 실패 시 부분 저장 데이터 정리를 시도하도록 구성했습니다.

단일 서버에서 서비스를 연결한 뒤에도, 각 단계의 성공과 전체 작업의 완료를 구분하고 **실패 후 남는 상태까지** 다뤄야 한다는 점을 배웠습니다.

03 구현 범위와 검증의 한계

부분 데이터 정리를 시도해도 정리 자체가 실패하면 데이터가 남을 수 있습니다. 또한 스레드 수와 배치 크기는 부하 시험으로 도출한 최적값이 아닌 초기 설정이었습니다.

이 한계를 정리하면서, 실패 대응 코드를 추가한 범위와 **실제로 보장하거나 검증한 범위를 구분**해 설명해야 한다는 점을 배웠습니다.

04 이전 프로젝트와 달라진 점

ACCIO에서는 프론트엔드 상태 관리와 팀의 백엔드 API 연동을 주로 담당했습니다. 이번에는 백엔드 역할을 직접 이해하고자 사용자와 문서 관리부터 분석 결과·답변 근거 저장까지 구성했습니다.

화면에서 서버 응답을 활용하던 경험을 바탕으로, 요청 이후의 **처리 흐름과 데이터 상태를 직접 설계**하는 범위까지 넓혔습니다.

자연어로 상품을 탐색하고 행동까지 연결하는 AI 쇼핑 서비스

자연어 상품 탐색과 대화형 추천 결과를 장바구니·주문·반품 같은 핵심 기능으로 연결

기간	형태	역할
2025.10 ~ 2025.11	5인 팀 프로젝트	Elasticsearch 검색 랭킹 · Frontend State Management · Guest Cart 병합 · 인증 흐름 개선

GitHub <https://github.com/seochanjin/Accio>

기획 배경

키워드 중심 쇼핑 검색은 사용자가 원하는 분위기·용도·취향을 구체적으로 표현하기 어려움
자연어로 요구사항을 입력하고 대화를 이어가며 상품을 좁혀가는 쇼핑 경험 구현

문제 정의

AI 검색 결과와 대화 상태가 화면 이동·새로고침 이후 소실 → 탐색을 처음부터 반복
비회원 장바구니와 로그인 후 장바구니 분리
인증 갱신 실패가 사용자 흐름 중단

핵심 가설

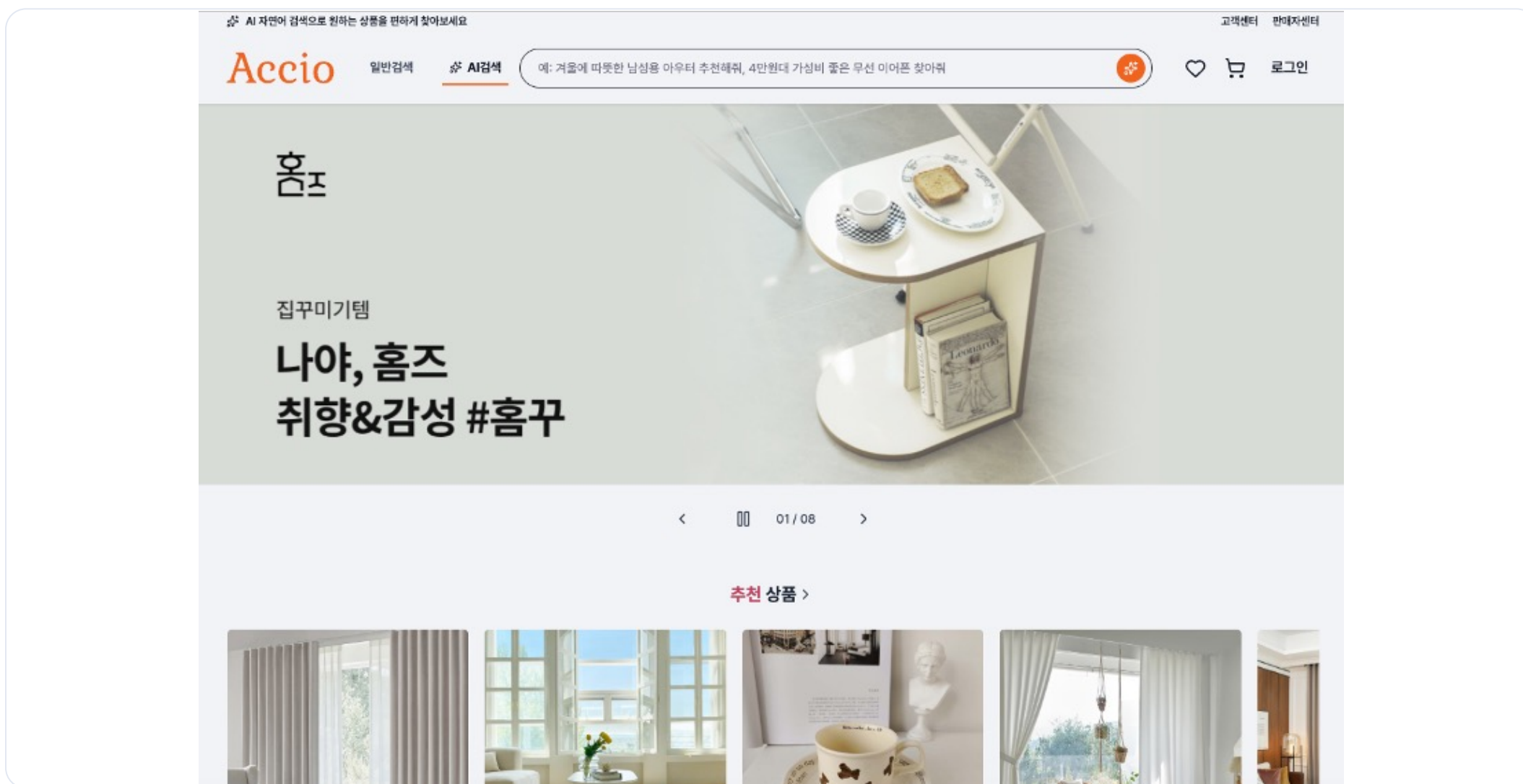
검색 조건과 AI 대화 상태를 세션 단위로 복원
비회원 장바구니를 로그인 계정에 병합
→ 탐색부터 구매까지 사용자 흐름 유지 가능

목표

AI 검색 세션 복원 · 비회원 장바구니 병합 · Refresh Token 기반 인증 흐름 적용
화면 이동·새로고침·로그인 전환 이후에도 탐색과 구매 연속성 확보

기술 스택

Frontend	React · TypeScript
Backend	FastAPI
AI / Search	AWS Bedrock · Titan Embedding · Elasticsearch
Data	MongoDB · Redis
Infrastructure	AWS



메인 화면 · 자연어 상품 검색과 추천 결과

AI 검색 세션 복원

검색 조건과 대화 결과를 세션 단위로 저장

화면 이동·새로고침 이후 복원

Guest Cart 병합

로그인 시 비회원 장바구니 항목을 계정 장바구니로 병합

인증 흐름 개선

Access Token 만료 시 Refresh Token으로 갱신

갱신 실패 시 인증 상태 정리

키워드와 의미 검색을 결합한 상품 탐색 구조

기존 Elasticsearch 키워드 검색에 Titan V2 임베딩 기반 KNN 검색을 추가

01 사용자 자연어 질의

“집에서 편하게 입을 따뜻한 옷” · “4만원대 가성비 좋은 무선 이어폰”
상품명뿐 아니라 용도·취향·가격 조건을 포함한 입력



02 Query 준비

규칙 기반 정규화 · Titan V2 Query Embedding · 1024차원
Redis Query Cache · Claude Fallback



03 Elasticsearch 단일 Hybrid 요청

Keyword Query

multi_match · term · match_phrase fuzziness · field boost

정확한 상품명·브랜드·카테고리 검색

KNN Query

Titan V2 Embedding k=50 · num_candidates=150

표현이 달라도 의미가 가까운 상품 검색

query + knn을 하나의 Elasticsearch 요청으로 실행



04 점수와 정렬 정책

Keyword function_score — reviewCount · rating · 기존 상품 rank 반영

Relevance: _score desc → rank asc

Popular: reviewCount desc → _score desc → rank asc



05 상품 결과

정확한 키워드 조건과 자연어 의미 조건을 하나의 결과 목록으로 반환

결과 부족 시 재검색

조건: 첫 페이지 요청 · 1차 검색 총 결과 10건 미만

Catalog Query Expansion → 확장 Keyword 생성 → 2차 Keyword 검색 → 결과가 더 많을 때만 교체

01 배경

- 기존 상품 검색은 Elasticsearch 키워드 검색 사용
- "집에서 편하게 입을 옷", "감성적인 홈 인테리어"처럼 상품명과 일치하지 않는 자연어 의도 반영 필요

02 문제

- 키워드 검색 — 상품명·브랜드·카테고리에 유리하나 다른 표현의 취향·용도는 놓침
- Vector 검색만 사용 — 정확한 상품명·문자열 조건의 영향 약화
- 두 방식 병행 유지 판단

03 설계 판단

- 키워드 검색을 대체하지 않고 Titan V2 Query Embedding 기반 KNN 검색 추가
- Keyword Query와 KNN Query를 하나의 Elasticsearch 요청으로 전달
- 검색 점수와 서비스 정렬 정책으로 최종 목록 구성

04 실제 적용

- Keyword Query — multi_match · term · match_phrase · fuzziness · 필드별 Boost
- function_score에 reviewCount · rating · 기존 상품 rank 반영
- KNN Query — Titan V2 1024차원 Query Embedding
- 첫 페이지 1차 검색 10건 미만 시 카탈로그 기반 Query 확장 후 2차 Keyword 검색

05 한계와 다음 검증

- Keyword-only · Vector-only · Hybrid 비교용 고정 평가셋 미구성
- RRF는 기능 제약으로 제거 → Elasticsearch 8.x 기본 Hybrid 동작 사용, 버전·설정에 따라 점수 결합 결과 상이 가능
- 평가셋 구성 후 Recall@K · MRR · nDCG@K · Zero-result Rate 비교 필요
- Embedding 호출 시간과 전체 검색 응답 시간 측정 필요

로그인 전환 과정에서도 장바구니와 인증 상태를 유지한 Frontend 흐름

비회원 장바구니를 로그인 계정에 병합하고, Access Token 만료 시 Refresh Token으로 인증 상태를 복구

01 Guest Cart 저장

비로그인 사용자가 상품 추가 → localStorage · cart-storage
productId · quantity · selectedColor · selectedSize · 상품 Snapshot

로그인 전 구매 상태를 브라우저에 유지



02 로그인 성공

사용자 인증 완료 로컬 장바구니 존재 여부 확인 — cart-storage 존재 + 상품 항목 1개 이상

로그인 완료 후 장바구니 병합을 후속 처리



03 병합 Payload 변환

상품 식별·옵션 productId · quantity selectedColor · selectedSize

상품 Snapshot priceSnapshot · nameSnapshot imageSnapshot

Frontend 상태를 Backend Cart API 형식으로 변환



04 서버 장바구니 병합 PUT /api/cart

성공

서버 장바구니 병합 → localStorage 제거 → 동일 상태 재전송 방지

실패

오류 기록 → 로그인 상태는 유지

Cart 병합을 로그인 완료와 분리된 후속 작업으로 처리



05 인증 상태 복구

Access Token 만료·부재 → Refresh Token 확인 → Access Token 재발급 요청

성공: 로그인 상태 유지 · 실패: 로그아웃 처리

Backend에 존재하던 재발급 흐름을 Frontend 인증 상태와 연결

01 배경

- 비회원이 담은 로컬 장바구니와 로그인 계정 장바구니의 저장 위치 상이
- 로그인 전 탐색·구매 준비가 로그인 이후에도 이어지도록 상태 전환 필요

02 문제

- 로그인 후 서버 장바구니만 조회 → 비회원 상태에서 담은 상품 소실 가능
- Access Token 만료 시 Refresh Token이 남아 있어도 Frontend에 재발급 흐름 부재 → 인증 상태 유지 실패

03 설계 판단

- 비회원 장바구니는 localStorage 유지, 로그인 성공 시 Backend Cart API 형식으로 변환해 전달
- 병합 작업을 인증 완료 이후의 후속 처리로 분리 → 병합 실패가 로그인 성공을 막지 않도록
- Access Token 만료 시 Refresh Token으로 재발급 시도하도록 연결

04 실제 적용과 결과

- 상품 ID · 수량 · 옵션 · 상품 Snapshot을 병합 Payload로 구성해 서버 전달
- 병합 성공 후 localStorage의 Guest Cart 제거 → 동일 상태 재전송 방지
- Access Token 부재·만료 시 Refresh Token으로 재발급, 성공 시 로그인 상태 유지
- 병합 오류는 기록하되 로그인 흐름은 계속 진행

05 한계와 다음 개선

- 서버·로컬 동일 상품의 수량 합산 · 옵션 충돌 · 중복 요청 정책을 명시적 도메인 계약으로 미정의
- 병합 API 멍등성, Refresh Token 만료·재발급 실패, 네트워크 오류 시 상태 전환 통합 테스트 부족
- 재설계 시 병합 규칙을 별도 도메인 정책으로 분리
- 로그인 · 병합 성공 · 병합 실패 · 토큰 만료 시나리오를 E2E 테스트로 검증

ACCIO / 상태 관리에서 배운 점과 백엔드로 이어진 관심

01 결과를 확인한 뒤 상태 변경

비회원 장바구니를 로그인한 계정에 병합하도록 요청하고, 성공한 뒤 로컬 데이터를 정리하는 흐름을 구현했습니다. 요청을 보냈다는 이유만으로 로컬 상태를 먼저 지우면 실패 시 기존 데이터를 잃을 수 있었습니다.

화면의 상태를 바꾸는 시점도 **서버의 처리 결과와 맞춰야** 한다는 점을 배웠습니다.

02 실패에 따라 이어지는 사용자 흐름

장바구니 병합 실패가 로그인 성공을 막지 않도록 후속 처리를 분리했습니다. 토큰이 만료되면 팀의 재발급 API와 연동해 인증 상태를 복구하도록 구현했습니다.

API를 연결하면서 오류가 발생했을 때 **어떤 상태를 유지하고 어떤 동작을 다시 시도할지** 구체적으로 정해야 한다는 점을 배웠습니다.

03 구현 후 남은 검증 과제

장바구니 병합과 인증 복구 흐름을 구현했지만, 병합 오류나 토큰 만료를 함께 다루는 통합 테스트는 충분하지 않았습니다. 동일 상품의 수량·옵션을 합치는 규칙도 명확히 정리할 필요가 남았습니다.

여러 기능이 이어지는 경우에는 화면과 서버가 기대하는 결과를 맞추고 **예외 경로까지 확인**해야 한다는 과제를 남겼습니다.

04 다음 프로젝트로 이어진 관심

프론트엔드를 전담하면서 일부 백엔드 코드도 수정했습니다. 한 파일에 많은 코드가 모인 구조를 보며, 요청과 데이터 처리 흐름을 이해하기 쉽게 구성하는 방법에 관심이 생겼습니다.

이후 Java와 Spring을 학습하고 NotebookLM Clone을 개인 프로젝트로 구현하며 **백엔드 전체 흐름을 직접 맡아봤습니다.**